```
  always executed code... ›
end
```

- Within the body of a method:

```
def method name and args
  code...
  code...
‹rescue ‹, parm ›* ‹, => var › ‹, then ›
    error handling code... ›*
‹else
    no exception code... ›
‹ensure
    always executed code... ›
end
```

- After the execution of a single statement:

```
statement ‹rescue statement ›*
```

A block or method may have multiple rescue clauses, and each rescue clause may specify zero or more exception parameters. A rescue clause with no parameter is treated as if it had a parameter of StandardError. This means that some lower-level exceptions will not be caught by a parameterless rescue class. If you want to rescue every exception, use this:

```
rescue Exception => e
```

When an exception is raised, Ruby scans the call stack until it finds an enclosing begin/end block, method body, or statement with a rescue modifier. For each rescue clause in that block, Ruby compares the raised exception against each of the rescue clause's parameters in turn; each parameter is tested using _parameter_ === $!. If the raised exception matches a rescue parameter, Ruby executes the body of the rescue and stops looking. If a matching rescue clause ends with => and a variable name, the variable is set to $!.

Although the parameters to the rescue clause are typically the names of exception classes, they can be arbitrary expressions (including method calls) that return an appropriate class.

If no rescue clause matches the raised exception, Ruby moves up the stack looking for a higher-level begin/end block that matches. If an exception propagates to the top level of the main thread without being rescued, the program terminates with a message.

If an else clause is present, its body is executed if no exceptions were raised in *code*. Exceptions raised during the execution of the else clause are not captured by rescue clauses in the same block as the else.

If an ensure clause is present, its body is always executed as the block is exited (even if an uncaught exception is in the process of being propagated).

Within a rescue clause, raise with no parameters will reraise the exception in $!.

Rescue Statement Modifier

A statement may have an optional rescue modifier followed by another statement (and by extension another rescue modifier, and so on). The rescue modifier takes no exception parameter and rescues StandardError and its children.